

# Informe Técnico: Simulador RV32I en Python

Simulación de Arquitectura de Computadores Basada en Módulos de  
Hardware (Verilog a Python)

Antonio

27 de junio de 2026

## Resumen

Este informe describe el diseño y desarrollo de un simulador educativo para el conjunto de instrucciones (ISA) RISC-V de 32 bits (RV32I). La estrategia de diseño se centró en la traducción funcional directa de los bloques constructivos de hardware descritos originalmente en Verilog hacia funciones estructuradas de Python. Este enfoque modular permite emular el comportamiento combinacional y secuencial ciclo a ciclo de una CPU monociclo (Single-Cycle). Se integró además una interfaz de desarrollo web (IDE) utilizando Next.js y componentes de shadcn/ui que proporciona depuración paso a paso (step-forward y step-back), visualización dinámica de registros, editor de memoria hexadecimal y desensamblado dinámico mediante Capstone. El backend utiliza FastAPI y ofrece autenticación con Google Login para control de accesos. El simulador fue validado con tres programas complejos de prueba, confirmando un comportamiento exacto al del hardware.

# Índice

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                | <b>3</b>  |
| <b>2. Estrategia de Diseño: De Verilog a Python</b>   | <b>3</b>  |
| 2.1. Mapeo de Módulos de Hardware . . . . .           | 3         |
| 2.2. El Ciclo de Reloj Simulado . . . . .             | 4         |
| <b>3. Estructura de Memoria y CPU</b>                 | <b>4</b>  |
| 3.1. Endianness y Operaciones de Memoria . . . . .    | 4         |
| <b>4. Instrucciones RV32I Soportadas</b>              | <b>5</b>  |
| <b>5. Arquitectura Tecnológica del Proyecto</b>       | <b>5</b>  |
| 5.1. Tecnologías del Frontend . . . . .               | 5         |
| 5.2. Tecnologías del Backend . . . . .                | 5         |
| <b>6. Resultados de Validación y Evidencias</b>       | <b>6</b>  |
| 6.1. Entorno de Trabajo e IDE General . . . . .       | 6         |
| 6.2. Inspección de Registros y Depuración . . . . .   | 7         |
| 6.3. Inspección de Memoria RAM . . . . .              | 7         |
| 6.4. Ejecución de Algoritmos Complejos . . . . .      | 8         |
| 6.5. Panel de Datapath y Señales de Control . . . . . | 9         |
| <b>7. Trabajo Futuro y Mejoras Propuestas</b>         | <b>10</b> |
| <b>8. Conclusiones</b>                                | <b>11</b> |

# 1. Introducción

El estudio práctico de la arquitectura de computadores requiere herramientas de simulación que ayuden a los estudiantes a comprender la interacción entre el camino de datos (datapath), la unidad de control y la memoria. Si bien las herramientas basadas en lenguajes de descripción de hardware (HDL) como Verilog o VHDL son el estándar de la industria, su curva de aprendizaje y la dificultad para integrarse con interfaces gráficas web dinámicas limitan su uso en laboratorios interactivos de software.

Este proyecto implementa un simulador monociclo del procesador RISC-V (RV32I) utilizando una estrategia novedosa: traducir de manera funcional todos los módulos de hardware escritos en Verilog directamente a módulos y scripts equivalentes en Python. El resultado es un simulador que emula el ciclo de reloj de hardware pero corre nativamente en un backend web liviano, acoplándose de manera interactiva a un frontend moderno.

---

## 2. Estrategia de Diseño: De Verilog a Python

Para mantener la fidelidad física y conceptual del procesador, no se utilizó un emulador abstracto a nivel de software. En su lugar, se tradujeron uno a uno los bloques lógicos de Verilog del procesador monociclo.

### 2.1. Mapeo de Módulos de Hardware

Cada archivo de hardware en Verilog se convirtió en una función puramente matemática/lógica en Python, conservando las señales de entrada y salida:

- **flopr (Registro de Estado):** Traducido a una función secuencial con señales de control de reset y carga. Representa el registro que mantiene el valor del PC.
- **mux2 (Multiplexor):** Simula la selección física entre dos buses de datos basada en una línea de control.
- **extend (Unidad de Extensión de Signo):** Decodifica los inmediatos empaquetados del formato RISC-V (I, S, B, U, J) y les aplica extensión de signo de 32 bits.
- **alu (Unidad Aritmético-Lógica):** Modela la ALU con sumadores, restadores, compuertas AND/OR/XOR y desplazadores lógicos/aritméticos (SLL, SRL, SRA).
- **control\_unit (Unidad de Control):** Decodifica los campos opcode, funct3 y funct7 de la instrucción y genera todas las señales de control de hardware (RegWrite, ALUSrc, MemWrite, ResultSrc, Jump, BranchType).
- **datapath (Camino de Datos):** El bloque combinacional principal que interconecta la ALU, los multiplexores, el banco de registros y el sumador de saltos del PC.

## 2.2. El Ciclo de Reloj Simulado

El bucle principal del simulador emula el flanco de subida del reloj (*rising edge*) en una función secuencial que repite cuatro fases secuenciales:

```
1 # 1. FETCH (Leer Instruccion de la Memoria ROM)
2 Instr = INSTRUCTION_MEMORY(PC)
3
4 # 2. DECODE (Unidad de Control genera senales combinacionales)
5 ImmSrc, RegWrite, ALUSrc, ALUControl, MemWrite, ResultSrc, Jump,
  BranchType = CONTROL_UNIT(Instr)
6
7 # 3. EXECUTE (El Datapath interconecta registros, ALU y calcula
  direcciones)
8 PCNext, ALUResult, WriteData, Zero, ImmExt, a3, PCPlus4, PCTarget, PCSrc =
  DATAPATH(
9   PC, Instr, ImmSrc, ALUSrc, ALUControl, ResultSrc, Jump, BranchType
10 )
11
12 # 4. MEMORY / WRITEBACK (Lectura/Escritura en RAM y escritura en Banco de
  Registros)
13 ReadData = DATA_MEMORY(ALUResult, WriteData, MemWrite, funct3)
14 Result = MUX2(ALUResult, ReadData, ResultSrc)
15 WRITE_REGISTER_FILE(a3, Result, RegWrite)
16
17 # Actualizacion del PC en el flanco del reloj
18 PC = FLOPR(PCNext, reset, width=32)
```

Listing 1: Bucle secuencial del ciclo de reloj simulado en Python

## 3. Estructura de Memoria y CPU

El simulador cuenta con dos memorias separadas:

1. **Memoria de Instrucciones:** Indexada en palabras de 32 bits, almacena el programa compilado. Se redimensiona de forma dinámica según el largo del programa.
2. **Memoria de Datos:** Representada por un bloque plano de **8 KB (8192 bytes)**. El direccionamiento es por byte (byte-addressable) permitiendo alineamientos exactos.

### 3.1. Endianness y Operaciones de Memoria

El simulador respeta la convención **Little-Endian** del estándar de RISC-V. La lectura y escritura física se maneja a nivel de bytes individuales:

- Cargas firmadas y no firmadas: **lb**, **lh**, **lw**, **lbu**, **lhu**.
- Guardados de datos: **sb**, **sh**, **sw**.

## 4. Instrucciones RV32I Soportadas

El conjunto de instrucciones emulado comprende la base de enteros del RISC-V (RV32I):

Cuadro 1: Instrucciones del ISA soportadas por el simulador

| Categoría           | Instrucciones                                        | Descripción               |
|---------------------|------------------------------------------------------|---------------------------|
| Tipo R              | add, sub, sll, slt, sltu, xor, srl, sra, or, and     | Operaciones Aritméticas   |
| Tipo I ALU          | addi, slti, sltiu, xori, ori, andi, slli, srli, srai | Aritmética con Inmediatos |
| Cargas (Loads)      | lb, lh, lw, lbu, lhu                                 | Lecturas Little-Endian    |
| Escrituras (Stores) | sb, sh, sw                                           | Escritura de bytes        |
| Branches (Tipo B)   | beq, bne, blt, bge, bltu, bgeu                       | Salto condicional         |
| Tipo U              | lui, auipc                                           | Carga de inmediato        |
| Tipo J y I Jumps    | jal, jalr                                            | Salto incondicional       |
| Sistema             | ecall                                                | Llamada al sistema        |

## 5. Arquitectura Tecnológica del Proyecto

El simulador se divide en una arquitectura desacoplada Cliente-Servidor (Frontend-Backend) moderna, interactiva y escalable.

### 5.1. Tecnologías del Frontend

El frontend está diseñado como una aplicación web de página única (SPA) estructurada de la siguiente manera:

- **Next.js 16.2 (React 19) y TypeScript:** Proporciona la estructura del IDE, el enrutamiento seguro de vistas y un tipado estático que evita errores en tiempo de ejecución.
- **shadcn/ui y Tailwind CSS:** Se emplearon componentes interactivos basados en Radix Primitives estilizados con Tailwind (como diálogos modal, tablas, acordeones, y botones) para dar una estética retro-futurista, consistente y profesional al IDE.
- **Google Login (OAuth 2.0):** Integración del flujo de autenticación federada de Google en el frontend para permitir accesos autenticados a las herramientas del simulador.

### 5.2. Tecnologías del Backend

El backend actúa como el motor de ejecución del hardware virtual y la API de control:

- **FastAPI (Python):** Framework de alto rendimiento utilizado para exponer servicios REST. Gestiona las sesiones interactivas de depuración almacenando el estado de la simulación en memoria.

- **Capstone Engine:** Usado para decodificar e inspeccionar en lenguaje natural los bytes que ejecuta la CPU virtual.
- **Base de Datos y Autenticación:** Soporte para persistencia en base de datos SQLAlchemy/PostgreSQL, encargado de validar las credenciales de sesión del Google OAuth y gestionar el perfil de usuario.

## 6. Resultados de Validación y Evidencias

A continuación se presentan las evidencias directas del funcionamiento del simulador capturadas desde la interfaz web final.

### 6.1. Entorno de Trabajo e IDE General

La interfaz del IDE proporciona una vista integrada del código fuente, la terminal virtual y los paneles de inspección del hardware.

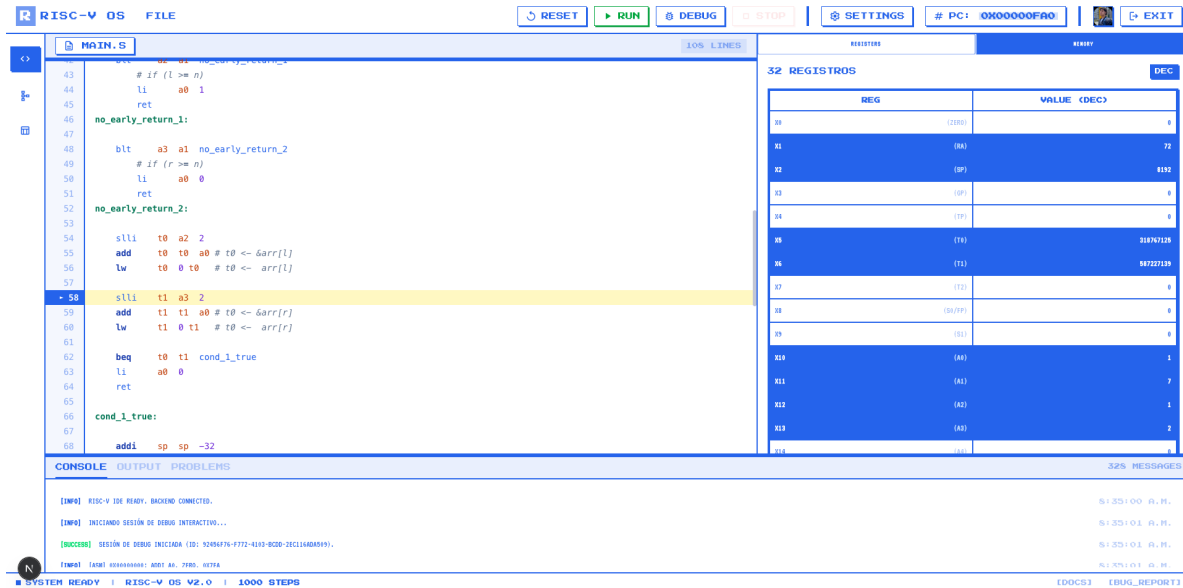


Figura 1: Vista general del IDE con código cargado y panel de registros en formato decimal.

El IDE permite configurar límites de pasos de simulación, formato de visualización (hexadecimal o decimal) y filtros para la visualización de registros con valor cero.

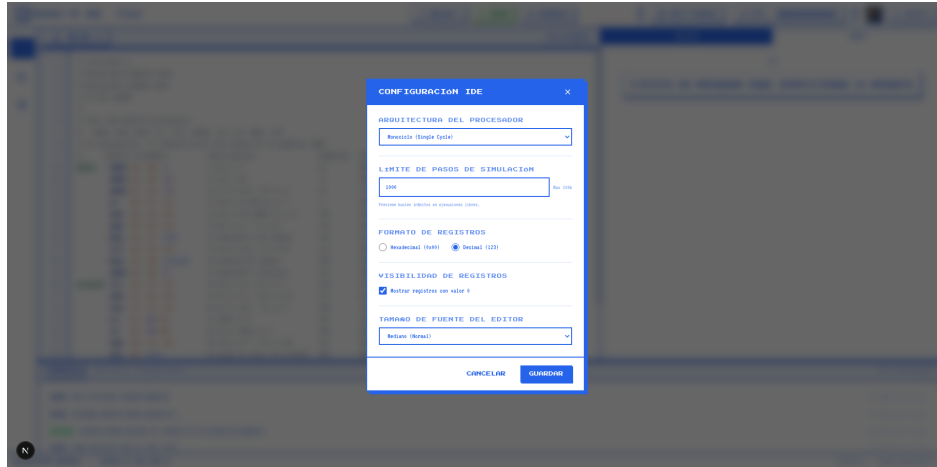


Figura 2: Modal de configuraciones del IDE (Settings).

## 6.2. Inspección de Registros y Depuración

Durante las pruebas se puede visualizar dinámicamente cómo varían los valores de los registros en base hexadecimal o decimal.

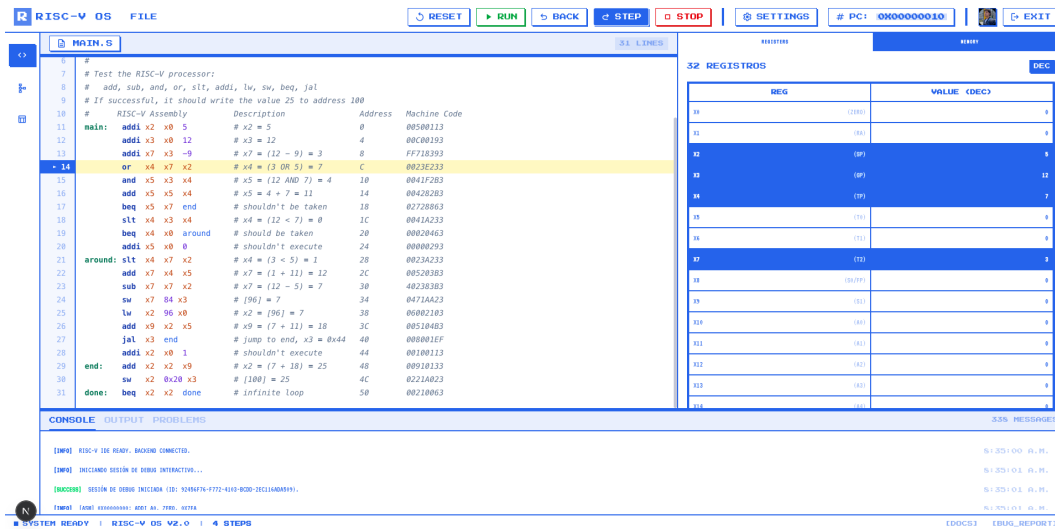


Figura 3: Panel de visualización de los 32 registros físicos del procesador en modo debug.

## 6.3. Inspección de Memoria RAM

El mapa de memoria permite examinar el contenido hexadecimal de la RAM en bloques de 16 bytes, indicando las etiquetas del PC y el SP correspondientes.



Figura 4: Mapa hexadecimal de memoria con indicadores visuales interactivos de PC y SP.

## 6.4. Ejecución de Algoritmos Complejos

Se validó la simulación con algoritmos recursivos que hacen uso intensivo de la pila de memoria y el banco de registros.

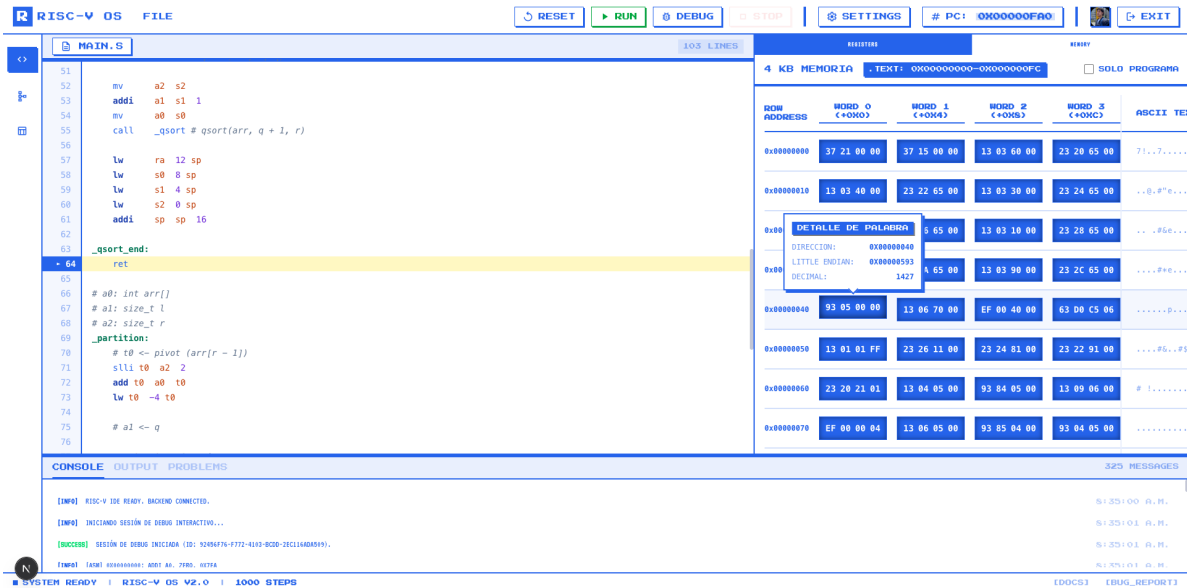


Figura 5: Validación exitosa del algoritmo Quicksort en el simulador, ordenando la memoria en 0x1000.



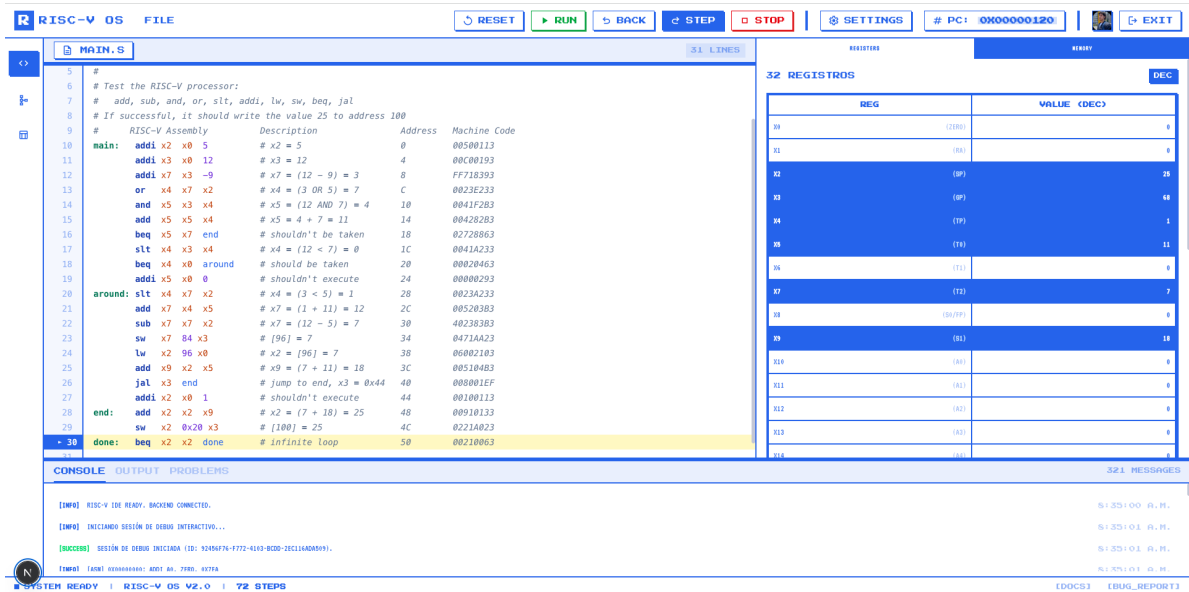


Figura 6: Simulación del algoritmo del árbol simétrico recursivo con valor final  $a0 = 1$ .

## 6.5. Panel de Datapath y Señales de Control

El sistema genera el diagrama del datapath y las señales de control correspondientes en tiempo real por cada instrucción ejecutada.

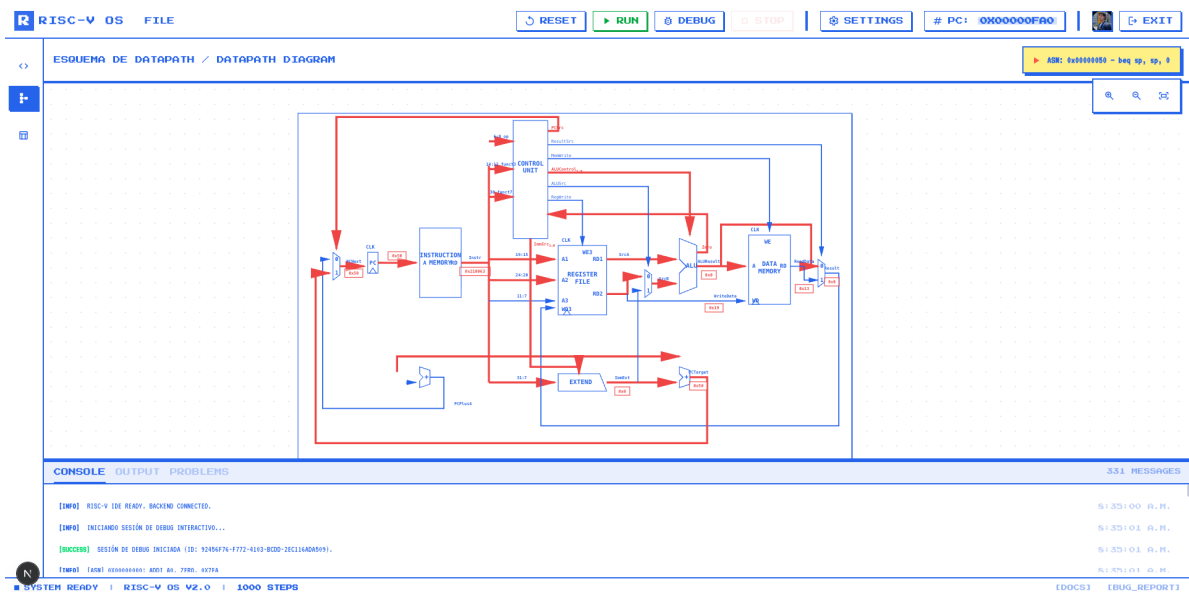


Figura 7: Diagrama esquemático interactivo del Datapath monociclo de la CPU.

La unidad de control expone sus señales combinatoriales para cada instrucción en una tabla de verdad interactiva.



Figura 8: Tabla de verdad de señales de control del ciclo actual.

## 7. Trabajo Futuro y Mejoras Propuestas

Aunque el simulador actual es 100 % funcional y cumple con todos los requerimientos, se proponen las siguientes mejoras de ingeniería para versiones futuras:

1. **Persistencia de Proyectos en Base de Datos:** Implementar el almacenamiento del código fuente y el historial de simulaciones de cada usuario en la base de datos (PostgreSQL) asociada a la cuenta de Google Login del estudiante, permitiendo guardar y cargar proyectos desde la nube.
2. **Soporte para Arquitecturas Multi-Cycle y Pipelined:** Expandir la lógica del backend para soportar la ejecución del simulador en modo Multi-Cycle (ejecución por microinstrucciones en múltiples ciclos de reloj) y Pipelined (segmentado con detección de riesgos de datos y control con instrucciones en vuelo).
3. **Exportación Detallada a PDF (Reporte de Ejecución):** Añadir un módulo que genere un reporte en PDF de tipo bitácora formal con la explicación paso a paso de las instrucciones ejecutadas, incluyendo volcados del estado del banco de registros y diagramas dinámicos del Datapath para cada ciclo de depuración.

## 8. Conclusiones

La traducción de descripciones de hardware en Verilog a implementaciones funcionales en Python demostró ser una excelente estrategia educativa y de ingeniería de software. Permite conservar la fidelidad de la arquitectura física monociclo, facilitando al mismo tiempo la depuración y desarrollo rápido en entornos web interactivos construidos con Next.js y FastAPI. La validación con algoritmos complejos y recursivos de alto consumo de pila (como Quicksort y árboles binarios) consolida la robustez de las unidades aritméticas, lógicas y del manejo little-endian de la memoria implementados.